

# Dialogue Summarization MVP

An AI-powered dialogue summarization feature that condenses multi-speaker group chat conversations into short, accurate summaries, built as part of a Flatiron School AI and Machine Learning project.

## Problem Statement and Business Context

Group messaging has become the default way people coordinate work and life, but high message volume turns long threads into a liability. Based on industry benchmarks used for planning this project:

- Users spend an average of **23 minutes per day** catching up on missed group threads.
- **41%** of users report missing an important detail in a busy conversation.
- Platforms see an **18% drop in daily engagement** among users who describe message volume as overwhelming.

Left unaddressed, this hurts user experience, retention, and competitive position against platforms that already offer AI-generated thread recaps.

**Proposed solution:** an automated dialogue summarization feature that sits on top of any conversation and produces a short, accurate summary of what was said, who said it, and what decisions or action items came out of it. The goal is not to replace reading the conversation, but to give people a fast way to decide whether they need to read it at all.

## Technical Approach and Methodology

- **Dataset:** SAMSum, a corpus of ~16,000 messenger-style conversations paired with human-written summaries. Loaded via the `(knkarthick/samsum)` community mirror, since the original `(samsum)` repository's loading script is deprecated and no longer resolves.

- **Model:** `t5-small`, a pre-trained encoder-decoder transformer, fine-tuned on the full SAMSum training set (14,731 examples) for 3 epochs.
- **Why encoder-decoder over decoder-only:** the encoder builds a complete, bidirectional representation of the full conversation before the decoder generates a single word of the summary, which produces more faithful, less repetitive summaries for this back-and-forth conversational structure than a decoder-only autoregressive approach.
- **Why T5-small over the originally scoped BERT-based setup:** T5-small is a smaller, faster pretrained checkpoint that made it possible to get a complete, working pipeline in place within the project timeline, while preserving the same architectural reasoning (encoder-decoder over decoder-only).
- **Optimization:** FP16 mixed precision and gradient accumulation, used to control memory and training time on a single GPU.
- **Preprocessing:** Each dialogue is prefixed with `summarize dialogue:` before tokenization to signal the task to the model. Speaker turns are preserved as-is in the input text, since who said what is part of what the model needs to summarize correctly.
- **Evaluation:** ROUGE-1, ROUGE-2, and ROUGE-L, benchmarked against a lead-3 extractive baseline (the first three lines of each dialogue used as a naive "summary"), plus manual qualitative review of generated summaries.

## Results and Evaluation

| Metric  | Extractive baseline | Fine-tuned model | Pitch target | Instructor-referenced target |
|---------|---------------------|------------------|--------------|------------------------------|
| ROUGE-1 | 32.52               | <b>47.79</b>     | ≥45 ✓        | >40 ✓                        |
| ROUGE-2 | 10.07               | <b>23.84</b>     | ≥20 ✓        | >15 ✓                        |

| Metric  | Extractive baseline | Fine-tuned model | Pitch target | Instructor-referenced target |
|---------|---------------------|------------------|--------------|------------------------------|
| ROUGE-L | 25.39               | 39.55            | ≥36 ✓        | >30 ✓                        |

The fine-tuned model clears every quantitative benchmark set for this project, beating the extractive baseline by a wide margin on every metric.

**Inference latency:** averaged **0.62 seconds** per conversation on CPU (max 1.05 seconds), well under the pitch's 2-second target, confirming the model is affordable to run without dedicated GPU infrastructure per request.

**Data characteristics:** 25.8% of training dialogues contain informal slang/shorthand and 8.4% contain emoji, quantifying the informal-language risk flagged during pitch feedback.

## Discussion of Limitations and Future Work

Passing every quantitative benchmark does not mean the underlying problem is fully solved. Manual review of generated summaries surfaced real, specific issues that ROUGE cannot detect:

- **Persistent speaker-attribution error:** across multiple training runs, one recurring example has the model reversing who performed an action (e.g., stating "Jerry will bring Amanda cookies" when the source conversation states the opposite). This error persisted even after training on the full dataset for 3 epochs, suggesting it is a harder edge case rather than something that resolves purely with more training data.
- **Occasional incoherent output:** a model-reload verification check produced a summary fragment that did not correspond to anything meaningful in the source dialogue, indicating the model can occasionally hallucinate rather than summarize.

- **ROUGE's blind spot:** a fluent summary with high word overlap to the reference can still be factually backwards. This project's evaluation process treats manual review as a required part of assessing quality, not an optional final check.

### Next steps:

- Make speaker turns more explicit in the input format (e.g., explicit turn markers) rather than relying on the model to infer conversational structure from a name and colon.
- Add an automated post-generation check that flags summaries with low semantic similarity to the source dialogue, to catch incoherent outputs before they reach a user.
- Investigate whether a larger checkpoint (T5-base) or a summarization-pretrained architecture (BART, PEGASUS) reduces attribution errors further.
- Expand the informal-language review into a labeled subset to check whether slang/emoji-heavy dialogues correlate with a higher rate of attribution errors.

### Repository Contents

- `project_3_dialogue_summarization_final.ipynb` — full notebook: data loading, preprocessing, model training, evaluation, sample outputs, and model save/reload verification.
- `README.md` — this file.

### Reproducing This Project

1. Open the notebook in Google Colab.
2. Set the runtime to GPU (Runtime → Change runtime type → L4 GPU or similar).
3. Run all cells top to bottom (Runtime → Run all). Package installation is handled in the setup cells.
4. Training on the full dataset for 3 epochs takes roughly 10-20 minutes on an L4 GPU.

## References

- Gliwa, B., Mochol, I., Biesek, M., & Wawer, A. (2019). SAMSum Corpus: A Human-annotated Dialogue Dataset for Abstractive Summarization.
- Raffel, C., et al. (2020). Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer (T5).
- Dataset mirror used: [knkarthick/samsum](https://huggingface.co/datasets/knkarthick/samsum) on Hugging Face